Artificial Intelligence Assignment

Problem Solving by Search: Uninformed (Blind) Search

Introduction

A lot of problems in AI can be turned into a question of finding a route. A robot looks for a way out of a room, a navigation app looks for the way from your house to a market, and a puzzle solver looks for the moves that turn a scrambled board into a solved one. In each case the computer has to look through many possible sequences of actions and pick one that reaches the goal. This idea is called **problem solving by search**.

Before any search can start, the problem has to be written down clearly. A search problem is usually described with five parts:

- **Initial state:** where the agent starts.
- **Actions:** what the agent is allowed to do in each state.
- **Transition model:** which state you end up in after doing an action.
- **Goal test:** a way to check whether a state is the goal.
- **Path cost:** a number that tells how expensive a path is (for example distance or time).

Searching then means building a **search tree**. The start state is the root, every action creates a child node, and the algorithm keeps picking a node from the **frontier** (the nodes that have been generated but not yet expanded) until it picks the goal. The only thing that separates one algorithm from another is *which node it picks next*.

Uninformed search (also called blind search) is the family of methods that have no extra hint about how close a state is to the goal. They only know the rules of the problem: they can generate children and test for the goal. Because they are blind, they are simple, but they may look at a lot of useless nodes. This assignment explains five uninformed strategies and runs each one on a small example.

1. Measuring a Search Algorithm

To compare the algorithms fairly, the textbook way is to judge them on four properties:

- **Completeness:** if a solution exists, will the algorithm always find it?
- **Optimality:** does it return the best (cheapest) solution?
- **Time complexity:** how many nodes does it generate?
- **Space complexity:** how many nodes must it keep in memory at once?

The symbols below are used in the rest of the report.

Symbol	Meaning
b	Branching factor: the maximum number of children a node can have.
d	Depth of the shallowest goal node.
m	Maximum depth of the search tree (it can be infinite).
l	The depth limit chosen for DLS.

C*	Cost of the optimal solution.
epsilon	Smallest step cost in the problem (assumed greater than 0).

2. The Example Used Throughout

To keep things easy to follow, I use one small tree for BFS, DFS, DLS and IDDFS. The start node is S and the goal is J. Children are always tried from left to right (alphabetical order). The goal sits at depth 3, so d = 3, the tree has a maximum depth of 3, and the branching factor is b = 3.

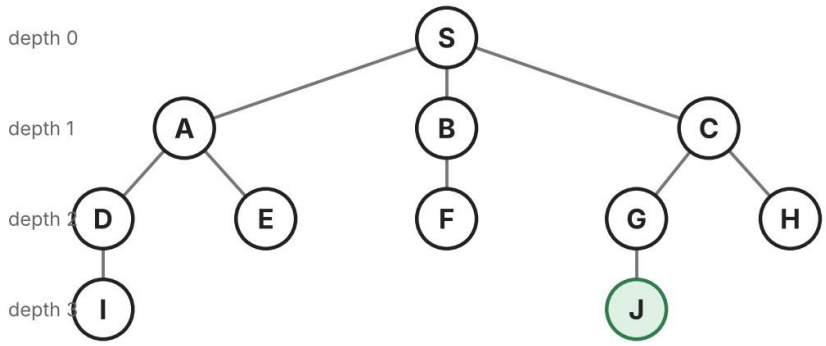


Figure 1: The example search tree. Start = S, goal = J (green).

For UCS the steps need different costs, so a separate small weighted graph is used in Section 8.

3. Breadth-First Search (BFS)

Idea

BFS explores the tree level by level. It expands the root, then all nodes at depth 1, then all nodes at depth 2, and so on. Because it always expands the shallowest unexpanded node, a **FIFO queue** (first in, first out) is used for the frontier. Think of ripples spreading out from a stone dropped in water.

Pseudocode

```
BFS(start, goal):     frontier =
queue containing start     explored
= empty set     while frontier is
not empty:
        node = frontier.dequeue()          # shallowest
node         if node == goal: return path to node
add node to explored         for each child of node:
if child not in explored and not in frontier:
                frontier.enqueue(child)
return failure
```

Trace on the example

Step	Node expanded	Queue after the step (front on the left)
1	S	A, B, C
2	A	B, C, D, E
3	B	C, D, E, F
4	C	D, E, F, G, H
5	D	E, F, G, H, I

6	E	F, G, H, I
7	F	G, H, I
8	G	H, I, J
9	H	I, J
10	I	J
11	J (goal found)	stop

BFS visits the nodes in the order S, A, B, C, D, E, F, G, H, I, J and returns the path **S - C - G - J**. It had to look at every node of depth 0, 1 and 2 plus most of depth 3 before it reached J.

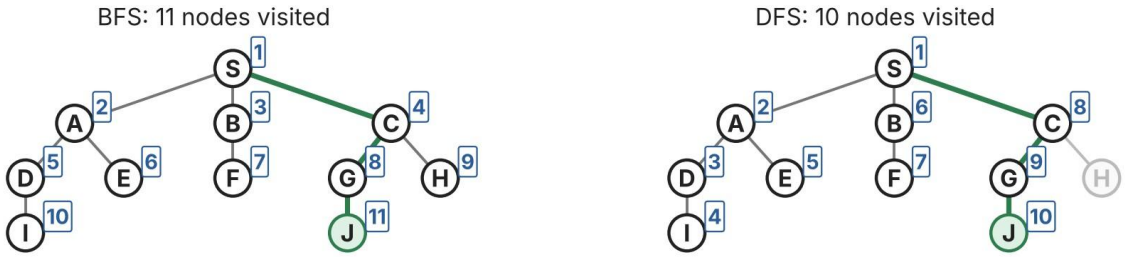


Figure 2: Expansion order of BFS (left) and DFS (right). The blue number on each node is the step at which it was visited. Grey nodes were never visited. The green path is the answer returned.

Properties

- **Complete:** yes, as long as the branching factor is finite.
- **Optimal:** yes when every step costs the same, because the shallowest goal is found first. If steps have different costs, BFS can return a path that is not the cheapest (see Section 8).
- **Time:** $O(b^d)$.
- **Space:** $O(b^d)$, since the whole bottom level may sit in the queue.

Advantages: it finds the path with the fewest steps and never gets stuck going down an endless branch. **Disadvantages:** memory use grows extremely fast. In practice BFS usually runs out of memory long before it runs out of time.

Where it is used: shortest path in unweighted graphs (for example the fewest moves in a maze), measuring "degrees of separation" in social networks, and web crawling level by level.

4. Depth-First Search (DFS)

Idea

DFS does the opposite of BFS. It always expands the *deepest* node in the frontier, so it goes down one branch as far as it can. Only when it hits a dead end does it back up and try the next branch. A **LIFO stack** (last in, first out) is used for the frontier. It can also be written as a simple recursive function, where the call stack does the job of the stack.

Pseudocode

```
DFS(start, goal):     frontier =
stack containing start     explored
= empty set     while frontier is
not empty:
```

```
        node = frontier.pop()               # deepest
node          if node == goal: return path to node
add node to explored          for each child of node
(right to left):              if child not in explored:
frontier.push(child)      return failure
```

Trace on the example

Children are pushed right to left so that the leftmost child is popped first. The top of the stack is on the left.

Step	Node expanded	Stack after the step (top on the left)
1	S	A, B, C
2	A	D, E, B, C
3	D	I, E, B, C
4	I	E, B, C
5	E	B, C
6	B	F, C
7	F	C
8	C	G, H
9	G	J, H
10	J (goal found)	stop

The order is S, A, D, I, E, B, F, C, G, J, so DFS went all the way down the left side (A, D, I) first. It also returned S - C - G - J, but that was partly luck: DFS does not look for the shortest path, it just returns the first goal it bumps into. Node H was never visited at all.

Properties

- **Complete:** no in general. In a tree with infinite depth, or a graph with loops and no repeated-state check, it can go down a branch forever. It is complete only in finite spaces with repeated-state checking.
- **Optimal:** no, because it may return a deep solution while a shallow one exists on another branch.
- **Time:** $O(b^m)$, which can be much worse than $O(b^d)$ if m is much bigger than d.
- **Space:** O(b x m). Only the current path and the unexplored siblings are stored, so memory is small.

Advantages: very low memory, and it can be quick when solutions are deep and plentiful. **Disadvantages:** can get lost in an infinite branch and does not guarantee a short answer.

Where it is used: maze solving, finding connected components, cycle detection, topological sorting and backtracking solvers such as Sudoku or N-Queens.

5. Depth-Limited Search (DLS)

Idea

DLS is DFS with a safety fence. We pick a depth limit *l*, and any node at depth *l* is treated as if it had no children. This fixes the infinite-branch problem of DFS. The price is that the limit has to be chosen well: if it is too small the goal is out of reach, and if it is too large we waste time.

DLS can end in three ways: it can **find the goal**, it can report a **cutoff** (the limit stopped it, so a solution might still exist deeper down), or it can report plain **failure** (the whole tree up to the limit was searched and there is no solution).

Pseudocode

```
DLS(node, goal, limit):     if node ==
goal: return path to node     if limit
== 0:   return CUTOFF
cutoff_happened = false     for each
child of node:
        result = DLS(child, goal, limit - 1)
if result is a path: return result         if
result == CUTOFF: cutoff_happened = true     if
cutoff_happened: return CUTOFF     return FAILURE
```

Trace on the example

With **l = 2** the search visits S, A, D, E, B, F, C, G, H. The nodes D and G are at depth 2 and still have children (I and J), but the limit forbids going below them, so the algorithm returns **cutoff**. The goal J is at depth 3, so it is missed. With **l = 3** the search visits S, A, D, I, E, B, F, C, G, J and finds the path S C - G - J.

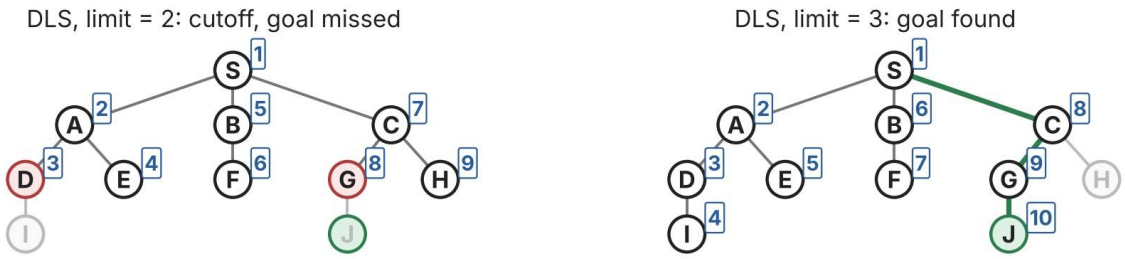


Figure 3: DLS with limit 2 (left) and limit 3 (right). Red nodes are the ones where the limit stopped the search even though they have children.

Properties

- **Complete:** only if l is at least d. If l < d, the goal cannot be reached.
- **Optimal:** no, for the same reason as DFS.
- **Time:** $O(b^l)$.
- **Space:** O(b x l).

Where it is used: when a sensible depth bound is already known (for example the number of moves allowed in a puzzle, or a game-tree search that must stop after a fixed number of moves), and as the building block of IDDFS.

6. Iterative Deepening Depth-First Search (IDDFS)

Idea

IDDFS removes the main weakness of DLS, which is guessing the limit. It simply runs DLS with limit 0, then limit 1, then limit 2, and so on, until the goal shows up. In this way it gets the **low memory** of DFS and the **shallowest-goal guarantee** of BFS together. The idea was described by Richard Korf in 1985.

It looks wasteful, because the upper levels are searched again and again. But in a tree most of the nodes are in the bottom level, so the repeated work is small compared with the last round.

Pseudocode

```
IDDFS(start, goal):
    for limit = 0, 1, 2, 3, ...:
        result = DLS(start, goal, limit)
        if result != CUTOFF: return result   # path or failure
```

Trace on the example

Limit	Nodes visited (in order)	Result	Count
0	S	cutoff	1
1	S, A, B, C	cutoff	4
2	S, A, D, E, B, F, C, G, H	cutoff	9
3	S, A, D, I, E, B, F, C, G, J	goal found: S - C - G - J	10

In total IDDFS visited 1 + 4 + 9 + 10 = 24 nodes here, against 11 for BFS. On a tiny tree the overhead looks big. For larger trees it is not: with b = 10 and d = 5, BFS generates about 111,111 nodes and IDDFS about 123,456, which is only around 11% more (I calculated these from the formulas, they are not measured values).

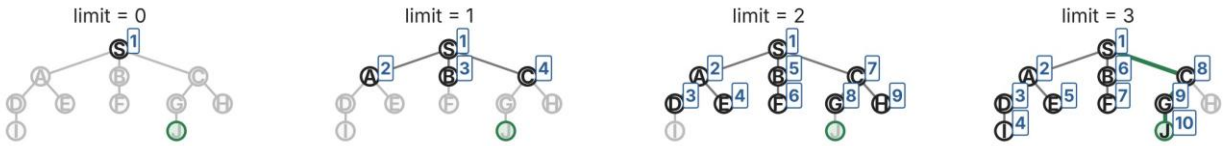


Figure 4: The four rounds of IDDFS on the example tree. Each round starts again from S with a larger depth limit until the goal is found in round 4.

Properties

- **Complete:** yes (finite branching factor).
- **Optimal:** yes when all steps have equal cost.
- **Time:** $O(b^d)$.
- **Space:** O(b x d).

Because of this mix of properties, IDDFS is commonly recommended as the default uninformed method when the search space is big and the depth of the solution is unknown. Game-playing programs also use the same "deepen step by step" idea so they can stop when the time is over and still hold a result from the last finished round.

7. Uniform-Cost Search (UCS)

Idea

So far, every step was treated as equal. Real problems are not like that: a road can be 1 km or 50 km long. UCS handles this by expanding the node with the **lowest path cost g(n)** so far, instead of the shallowest one. The frontier is a **priority queue** ordered by g(n). If all step costs are equal, UCS behaves just like BFS. It is closely related to Dijkstra's shortest-path algorithm.

Two details matter. First, the goal test is done when a node is *expanded* (taken out of the queue), not when it is generated, because a cheaper way to the goal may still be waiting in the queue. Second, if a node is reached again by a cheaper path, the cheaper path replaces the old one.

Pseudocode

```
UCS(start, goal):     frontier = priority queue ordered by path cost g,
containing (start, 0)     explored = empty set     while frontier is not
empty:
        node = frontier.pop_lowest_cost()         if
node == goal: return path to node and its cost
add node to explored         for each (child,
step_cost) of node:             new_cost = g(node) +
step_cost             if child not in explored and not
in frontier:
                frontier.insert(child, new_cost)
else if child in frontier with a higher cost:
                replace it with new_cost
return failure
```

Example graph

The graph in Figure 5 has start S and goal G. The numbers on the edges are the costs of the steps. The left picture shows the path BFS would return (fewest steps) and the right picture shows the UCS path (lowest total cost).

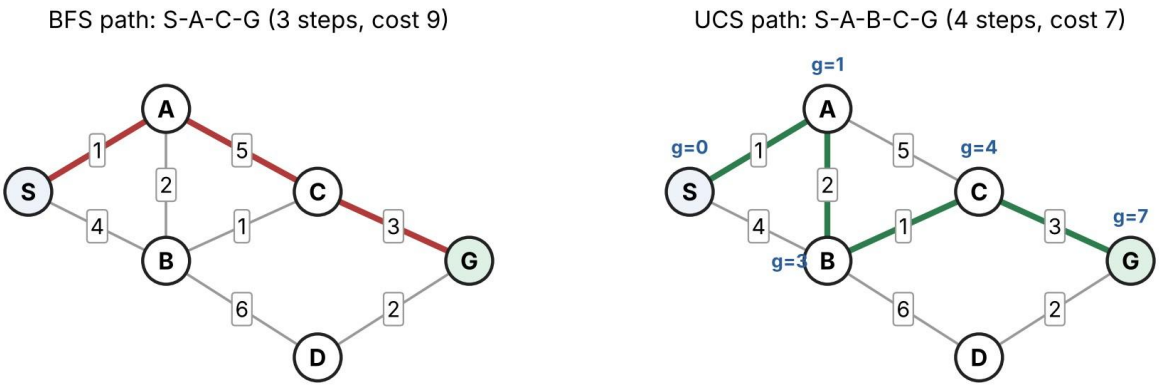


Figure 5: The same graph solved two ways. BFS finds S - A - C - G with 3 steps but a cost of 9. UCS finds S - A - B - C - G with 4 steps and a cost of only 7. The blue g values are the costs UCS recorded.

Trace

Step	Node expanded (g)	Frontier after the step	Note
1	S (0)	A:1, B:4	start node
2	A (1)	B:3, C:6	B improved from 4 to 3 via A
3	B (3)	C:4, D:9	C improved from 6 to 4 via B
4	C (4)	G:7, D:9	G is generated but not yet tested
5	G (7)	stop	goal expanded: cost 7, path S-A-B-C-G

Notice step 4. G enters the queue with cost 7, but UCS does not stop yet, because the goal test happens only when G is taken out. Here nothing cheaper was left in the queue (D has cost 9), so G is expanded next and the answer is confirmed. I checked the three other simple routes by hand: S-B-C-G costs 8, S-A-C-G costs 9 and S-A-B-D-G costs 11, so 7 is indeed the cheapest.

Properties

- **Complete:** yes, provided every step cost is at least some small positive number epsilon.
- **Optimal:** yes, because nodes are expanded in order of increasing path cost.
- **Time and space:** $O(b^{1 + floor(C^*/epsilon)})$, which can be much larger than b^d when there are many cheap steps.

Advantages: finds the cheapest path even when step costs differ. **Disadvantages:** it explores in all directions around the start (like BFS), so it can expand many nodes that point away from the goal, and it needs memory for the whole frontier. Zero-cost steps can make it loop forever.

Where it is used: route finding with distances or travel times, network routing and any problem where actions have different costs.

8. Comparison of the Five Algorithms

Method	Frontier	Complete?	Optimal?	Time	Space
BFS	FIFO queue	Yes (finite b)	Yes, if step costs are equal	$O(b^d)$	$O(b^d)$
DFS	LIFO stack	No (infinite depth or loops)	No	$O(b^m)$	O(bm)
DLS	LIFO stack + limit	Only if l >= d	No	$O(b^l)$	O(bl)
IDDFS	LIFO stack, repeated	Yes (finite b)	Yes, if step costs are equal	$O(b^d)$	O(bd)
UCS	Priority queue	Yes (step cost >= epsilon > 0)	Yes	$O(b^{1+floor(C^*/epsilon)})$	$O(b^{1+floor(C^*/epsilon)})$

The big difference in memory is easy to see in numbers. For a branching factor of 2, the next chart compares the rough number of nodes BFS has to hold with the rough number held by the depth-first family. This is only a simple estimate from the formulas above, but the trend is the real point: one line explodes and the other barely moves.

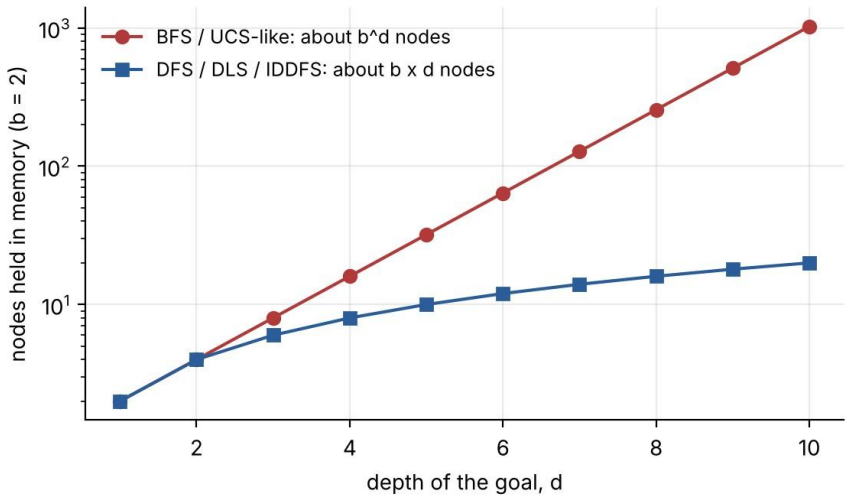


Figure 6: Approximate memory need for b = 2 (log scale). BFS grows as b^d, while DFS, DLS and IDDFS grow roughly as b x d.

Which one should I pick?

- All steps cost the same and memory is not a problem: **BFS**.
- Memory is tight and any solution will do: **DFS**.
- A good depth bound is already known: **DLS**.
- Big search space and unknown solution depth: **IDDFS**.
- Steps have different costs and the cheapest path is needed: **UCS**.

9. Conclusion

Uninformed search algorithms are the starting point of problem solving in AI. They differ only in the order in which they pick nodes from the frontier, but that small change decides everything: BFS is safe and shortest in steps but memory hungry, DFS is light but can wander forever, DLS fixes the wandering with a limit that has to be guessed, IDDFS removes the guess by repeating DLS with growing limits, and UCS cares about real costs rather than the number of steps. None of them knows anything about where the goal is, so on large problems they waste effort. That is the reason informed (heuristic) methods such as A* are the next topic after this one.